



From Monolith to Microservices

Defining the Monolith



Single-process system

All code runs in one process



Distributed monolith

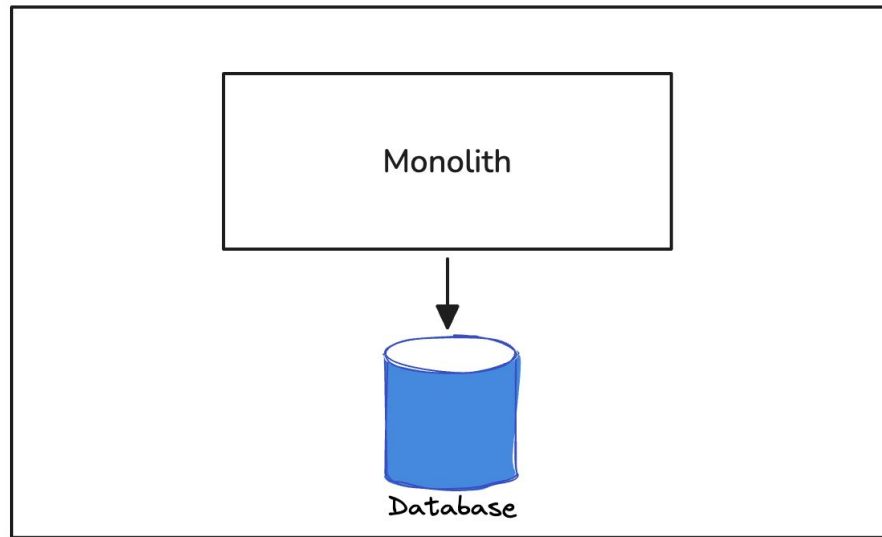
Multiple services that must be deployed together



Third-party black-box systems

Software developed by others (SaaS, CRM, HR systems)

The Single Process Monolith



A single-process monolith: all code is packaged into a single process

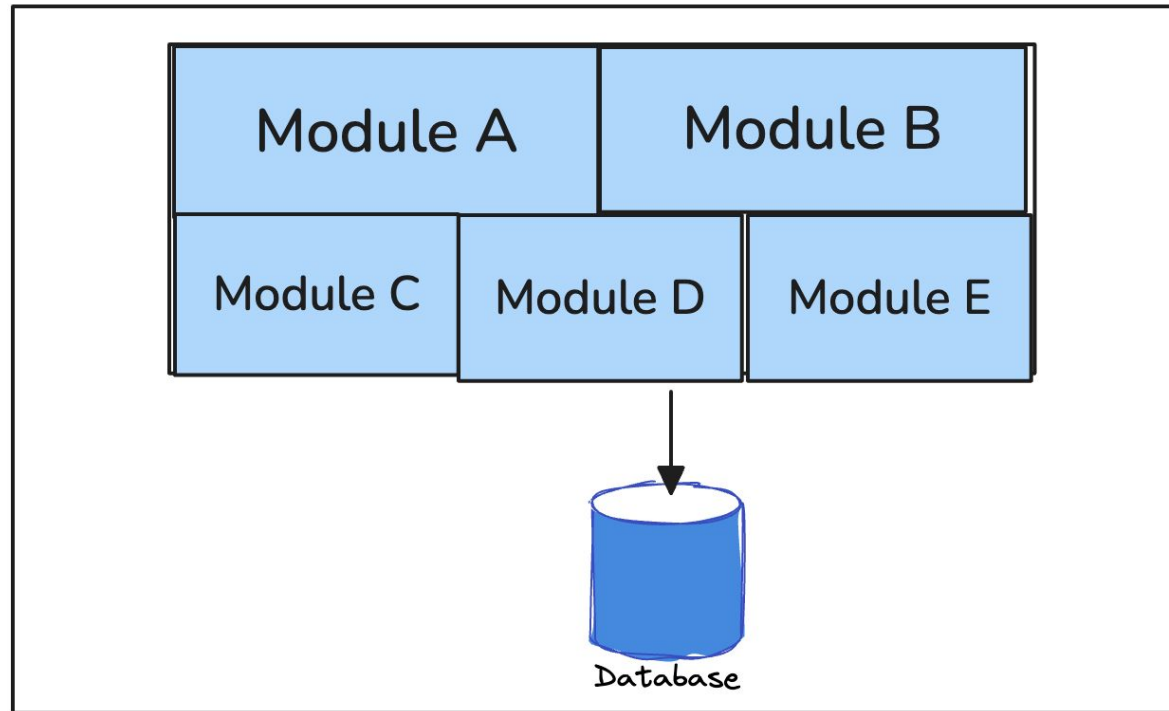
The modular monolith is a subset of the single process model.



Single Process Monolith

All code runs as a single,
indivisible unit.

Modular Monolith



A modular monolith with a decomposed database



Modular Monolith

Independent modules
combined for a single
deployment.

The Distributed Monolith

The worst of both worlds: distributed complexity without independent deployment

“Distributed monoliths have all the disadvantages of a distributed system, and the disadvantages of a single-process monolith, without having enough upsides of either.”

— Sam Newman

Third-Party Black-Box Systems



Definition

Software developed by others where you cannot change the code, such as payroll, CRM, or HR systems.



Types

These can be off-the-shelf software or SaaS products.



Decomposition

Many decomposition techniques can still be used even when you cannot change the underlying code.

Decomposing software you don't own
or control

Three-tiered Architecture

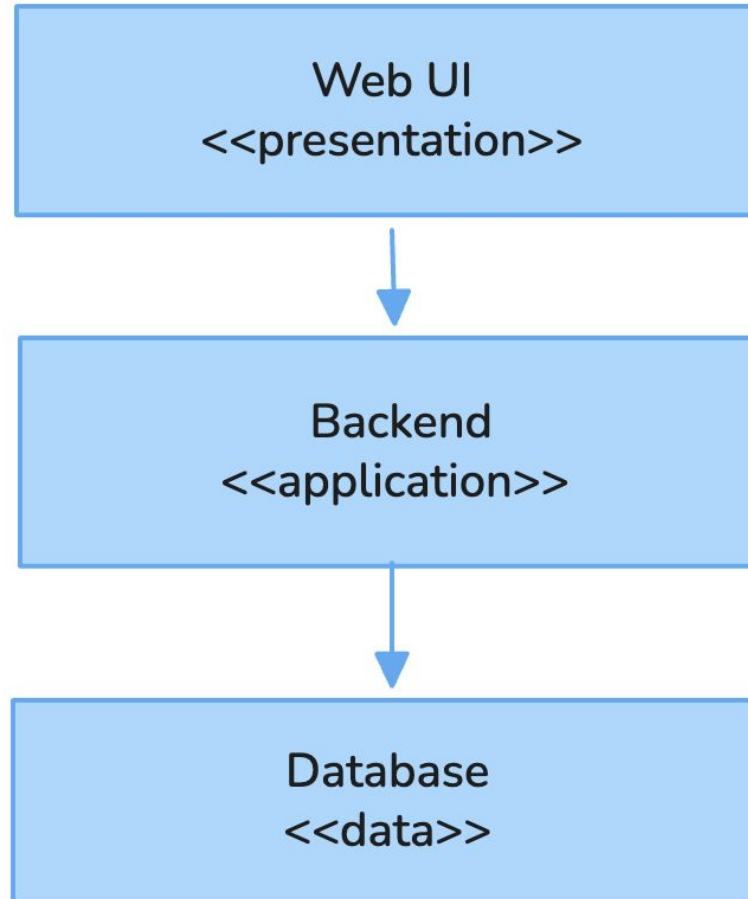
The three-tiered architecture is so common partly because it is universal—everyone's heard about it. The biggest reason we see this architecture again and again is because it is based on how we organize our teams.

“Any organization that designs a system...will inevitably produce a design whose structure is a copy of the organization’s communication structure.”

— Melvin Conway

In the past, IT organizations grouped people by core competency: database admins together, Java developers together, frontend developers together.

Our aspirations have changed. We now group people in poly-skilled teams, to reduce hand-offs and silos.

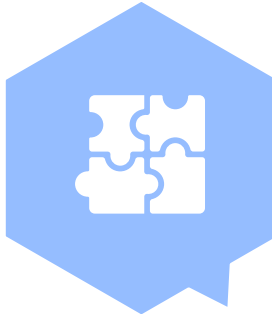


SomNOG Event Management System as a traditional tree-tiered architecture

Advantages of Monoliths

Easy code reuse

all code is available in one place to use



Simpler deployment

avoids pitfalls of distributed systems

Simplified testing

end-to-end testing is greatly simplified



Streamlined workflows

easier monitoring and troubleshooting

Challenges of Monoliths

1 Scaling Inefficiency

Full-stack duplication:

You must scale the entire application even if only a single feature experiences high traffic.

Resource waste:

Running multiple instances of the entire monolith consumes excessive memory and CPU for components that do not need it.

Database bottlenecks:

All features share a single database, often leading to resource contention and performance slowdowns during peak hours.

2 Technology Lock-in

Single stack constraints:

The entire system is bound to the initial language and framework chosen at the start of the project.

Difficult upgrades:

Upgrading a core framework or library is extremely risky and labor-intensive because it impacts every part of the application.

Hiring barriers:

It becomes harder to adopt modern, more efficient tools or attract talent who want to work with newer technologies.

What are Microservices?

Microservices also have the advantage of being technology agnostic.

Microservices are independently deployable services modeled around a **business domain**. They communicate with each other via networks, and as an architecture choice offer many options for solving the problems you may face. It follows that a microservice architecture is based on multiple collaborating microservices.

Independent Deployability

Independent deployability is the idea that we can make a change to a microservice and deploy it into a production environment without having to utilize any other services.

To guarantee independent deployability, we need to ensure our services are **loosely coupled**—in other words, we need to be able to change one service without having to change anything else. This means we need explicit, well-defined, and stable contracts between services.

Some implementation choices make this difficult—the sharing of databases, for example, is especially problematic. The desire for loosely coupled services with stable interfaces guides our thinking about how we find service boundaries in the first place.

Modeled Around a Business Domain

Making a change across a process boundary is expensive. If you need to make a change to two services to roll out a feature, and orchestrate the deployment of these two changes, that takes more work than making the same change inside a single service (or, for that matter, a monolith).

It therefore follows that we want to find ways of ensuring we make cross-service changes as infrequently as possible.

Own Their Own Data

One of the things people have the hardest time with is the idea that microservices should not share databases. If one service wants to access data held by another service, then it should go and ask that service for the data it needs.

“

“Don’t share databases, unless you really have to. And even then do everything you can to avoid it. In my opinion, it’s one of the worst things you can do if you’re trying to achieve independent deployability.”

Technology

It can be all too tempting to grab a whole load of new technology to go along with your shiny new microservice architecture, but I strongly urge you not to fall into this temptation. Adopting any new technology will have a cost—it will create some upheaval.

Microservices are fundamentally technology agnostic. As long as your services can communicate with each other via a network, everything else is up for grabs.

You don't have to use Kubernetes, Docker, containers, or the public cloud. You don't have to code in Go or Rust or whatever else.

If you know PHP best, start building services with PHP! Choose the approach that works for you, and change things to address problems as and when you see them.

What Advantages Can Microservices Bring?

The advantages of microservices are many and varied. The independent nature of the deployments opens up new models for improving the scale and robustness of systems, and allows you to mix and match technology.

As services can be worked on in parallel, you can bring more developers to bear on a problem without them getting in each other's way. It can also be easier for those developers to understand their part of the system, as they can focus their attention on just one part of it.

Process isolation also makes it possible for us to vary the technology choices we make, perhaps mixing different programming languages, programming styles, deployment platforms, or databases to find the right mix.

Perhaps, above all, microservice architectures give you flexibility. They open up many more options regarding how you can solve problems in the future.

What Problems Do They Create?

Because services can be deployed in different computers / servers, one of the main challenges is the way in which these computers talk to each other: networks.

Communication between computers over networks is not instantaneous.

This means we have to worry about latencies—and specifically, latencies that far outstrip the latencies we see with local, in-process operations.

Things get worse when we consider that these latencies will vary, which can make system behavior unpredictable. And we also have to address the fact that networks sometimes fail—packets get lost; network cables are disconnected.

Size

“How big should a microservice be?”

is probably the most common question. However, when you get into what makes microservices work as a type of architecture, the concept of size is actually one of the least interesting things.

How do you measure size? Lines of code?

Something that might require 25 lines of code in Java could possibly be written in 10 lines of Clojure. Some languages are more expressive than others.

Ultimately, the concept of “size” is highly contextual. It depends on who you ask and how long they've worked with the system.

On Coupling and Cohesion

Cohesion

"the code that changes together, stays together."

Coupling

The more things are "coupled" the more they have to change together.

Constantine's Law:

"A structure is stable if cohesion is high, and coupling is low."

— Larry Constantine

High Cohesion:

each part has one clear, specific job. Everything inside that part belongs together.

Low Coupling:

the different parts don't depend heavily on each other. If you change or fix one part, it won't accidentally break something else.

Think of it like a team of specialists: a soccer team is stable because every player knows their specific role (high cohesion), but if one player gets substituted, the whole team doesn't collapse (low coupling).

High Cohesion (Focus on One Job)

Each microservice should only care about its own tightly related data and tasks. Things that belong together stay together.

The Registration & Applicant Service

This service focuses entirely on managing attendees, ticket forms, and sign-ups. It doesn't contain code for finding workshop rooms or processing sponsor logos.

The Sponsor Service

This service only handles sponsor tiers, logos, and invoices. Because it has high cohesion, a developer looking at this code will only see sponsor-related logic.

Low Coupling (Independence)

The services must be independent. Changing or breaking one service should not crash the others.

Low Coupling

The Event & Workshop Services

If the Workshop Service goes down because a facilitator updates a room assignment, the Registration Service should keep working perfectly. Users should still be able to buy a general ticket to the overall event.

Low Coupling

The Facilitator Service

If you completely rewrite how facilitators log into the system, you shouldn't have to change any code inside the Sponsor Service.

The Direct Comparison

High Cohesion:

**The Sponsor Service only
does sponsor things.**

Low Coupling:

**If the Sponsor Service
crashes, people can still
register for Workshops.**

References

Building Microservices — Sam Newman

Monolith to Microservices — Sam Newman